

Lecture 06: Deciphering File Formats

August 26, 2024

Overview

- ▶ Walkthrough of Data Reverse Engineering

Interoperability

- ▶ Data Interoperability is a bigger deal than module interoperability
- ▶ Users often like to share data
- ▶ This often requires both to use the same software or even operating system
- ▶ Goals of data reverse engineering:
 - ▶ Read a file format
 - ▶ Generate a file format (often more difficult than reading)
 - ▶ Monitor a network protocol
 - ▶ Speak a network protocol
 - ▶ Read in-memory data, e.g., map hacking, cheating, etc.

Walkthrough - Cryptex

- ▶ Program contrived by the author to demonstrate reversing a file format
- ▶ Essentially a custom format akin to a password protected .zip file
- ▶ Uses 3DES to protect each file
- ▶ Protection is commensurate to the password used – no back doors

Using Cryptex

It's always a good idea to begin reversing a program by using it and getting a feel for how it works. You want to be familiar with its interface, because certain elements will give you clues about where to search for the implementation of particular features.

- ▶ Command-line interface – no GUI
- > `Cryptex.exe`
- ▶ Displays its help / manual

Commands

- ▶ Add / encrypt
- ▶ Extract / decrypt
- ▶ List
- ▶ Delete

This particular demonstration of reversing a file format also helps us audit the protection provided by the program.

- ▶ Could be intentional backdoor
- ▶ Quality could be poor – unintentional backdoor
 - ▶ Uses a custom crappy algorithm
 - ▶ Misuses strong commercial algorithm

Reversing Cryptex - Looking at the files

- ▶ Create simple, small files with identifiable values
 - ▶ Not always possible depending on the provided program
 - ▶ Could only offer to read a file
- ▶ Check for obvious things in the resulting file
 - ▶ Plain password? – please no!
 - ▶ Encrypted file name, or just contents?
 - ▶ File size?
- ▶ Try breaking some rules and see if checks are implemented
 - ▶ Can I list files without a password or the wrong password?

Examining imports

- ▶ Open it in a hex editor
 - ▶ See the magic number
 - ▶ Check for anything in plain sight
 - ▶ Check character distribution
- ▶ Can't get much further without some code RE
- ▶ Dump its imported functions
- > `dumpbin /IMPORTS Cryptex.exe`
 - ▶ Not everything listed here is called directly
 - ▶ Could be a transitive dependency from static linking
 - ▶ Windows Crypto API is like J2SE for Java
 - ▶ Its use of the API will expose a lot about how it uses encryption

Breaking on errors

- ▶ Breakpoint on `printf`
 - ▶ Careful which one: `msvcrt` or `MSVCR71`?
- ▶ Step until you catch the print for “Invalid”
- ▶ How did you get here?
- ▶ Where was the decision made?

Breaking on memory access

- ▶ Restart the program
- ▶ Hardware breakpoint on 00405038
- ▶ Step up to the call from Cryptex
- ▶ Find the sequence of calls setting up the hash
- ▶ In particular, CryptCreateHash

Understanding API calls

- ▶ Lookup CryptCreateHash up on MSDN
- ▶ Notice a parameter called ALG_ID
- ▶ What value does this parameter get?
- ▶ What does that value mean?

Understanding API calls

- ▶ Lookup CryptCreateHash up on MSDN
 - ▶ Notice a parameter called ALG_ID
 - ▶ What value does this parameter get?
 - ▶ What does that value mean?
-
- ▶ Now, what is it hashing?
 - ▶ What API call would reveal this?
 - ▶ What is this data?
 - ▶ Where is this data?

Tracing data flow

- ▶ Why can't we repeat the same method as 00405038?
- ▶ WinDbg's stack unwind seems broken, let's step out carefully.
- ▶ What's this function hashing?
- ▶ What else is it doing?

Tracing data flow

- ▶ Why can't we repeat the same method as 00405038?
- ▶ WinDbg's stack unwind seems broken, let's step out carefully.
- ▶ What's this function hashing?
- ▶ What else is it doing?

- ▶ Why the second hash? Why not store the first in the header?
- ▶ Why the first hash?

Breaking on ReadFile

- ▶ Let's you step from the point data enters the flow
- ▶ What's the issue if you approach this naïvely?
- ▶ How might you be more careful in the approach?
- ▶ Let's invoke `cryptex 1` to learn how the directory is stored.

Breaking on ReadFile

- ▶ Let's you step from the point data enters the flow
 - ▶ What's the issue if you approach this naïvely?
 - ▶ How might you be more careful in the approach?
 - ▶ Let's invoke `cryptex 1` to learn how the directory is stored.
-
- ▶ Entry sizes usually given away by loop increments
 - ▶ Can deal with high loop counts by “stepping to”
 - ▶ Alternatively, setting a breakpoint after the loop

Dealing with stack variables

- ▶ Set a breakpoint on `CreateFileW`
- ▶ Invoke `cryptex x`
- ▶ Some parameters cannot be seen clearly on the stack
- ▶ Record the address(es) of the call(s)
- ▶ Restart and set them as breakpoints for a better look
- ▶ Be sure to consider `PUSHes`, etc., involved.
- ▶ Which is the call to open the extracted file?

Interpreting `call; test eax, eax`

- ▶ For most Windows calls
- ▶ return value of type `BOOL` indicating success
- ▶ Can likely see which block is the error condition by call to `GetLastError`
- ▶ Or can read the MSDN and check the jump condition